



Introduction to Computers, the Internet and Visual C#

Chapter 1 of Visual C# How to Program, 6/e

OBJECTIVES

In this chapter you'll:

- Learn basic computer hardware, software and data concepts.
- Be introduced to the different types of computer programming languages.
- Understand the history of the Visual C# programming language and the Windows operating system.
- Learn what cloud computing with Microsoft Azure is.
- Understand the basics of object technology.
- Be introduced to the history of the Internet and the World Wide Web.
- Understand the parts that Windows, .NET, Visual Studio and C# play in the C# ecosystem.
- Test-drive a Visual C# drawing app.

1.1 Introduction

1.2 Computers and the Internet in Industry and Research

1.3 Hardware and Software

1.3.1 Moore's Law

1.3.2 Computer Organization

1.4 Data Hierarchy

1.5 Machine Languages, Assembly Languages and High-Level Languages

1.6 Object Technology

1.7 Internet and World Wide Web

1.8 C#

1.8.1 Object-Oriented Programming

1.8.2 Event-Driven Programming

1.8.3 Visual Programming

1.8.4 Generic and Functional Programming

1.8.5 An International Standard

1.8.6 C# on Non-Windows Platforms

1.8.7 Internet and Web Programming

1.8.8 Asynchronous Programming with **async** and **await**

1.8.9 Other Key Programming Languages

1.8 C#

- 1.8.1 Object-Oriented Programming
- 1.8.2 Event-Driven Programming
- 1.8.3 Visual Programming
- 1.8.4 Generic and Functional Programming
- 1.8.5 An International Standard
- 1.8.6 C# on Non-Windows Platforms
- 1.8.7 Internet and Web Programming
- 1.8.8 Asynchronous Programming with **async** and **await**
- 1.8.9 Other Key Programming Languages

1.9 Microsoft's .NET

- 1.9.1 .NET Framework
- 1.9.2 Common Language Runtime
- 1.9.3 Platform Independence
- 1.9.4 Language Interoperability

1.10 Microsoft's Windows® Operating System

1.11 Visual Studio Integrated Development Environment

1.12 **Painter** Test-Drive in Visual Studio Community

1.1 Introduction

- ▶ There are billions of personal computers in use and an even larger number of mobile devices with computers at their core.
- ▶ Since it was released in 2001, C# has been used primarily to build applications for personal computers and systems that support them.
- ▶ The explosive growth of mobile phones, tablets and other devices also is creating significant opportunities for programming mobile apps.



1.2 Computers and the Internet in Industry and Research

- ▶ Figure 1.1 provides a few examples of the ways in which computers are improving people's lives in research, industry and society.

Name	Description
Electronic health records	These might include a patient's medical history, prescriptions, immunizations, lab results, allergies, insurance information and more. Making these available to health-care providers across a secure network improves patient care, reduces the probability of error and increases the health-care system's overall efficiency, helping control costs.
Human Genome Project	The Human Genome Project was founded to identify and analyze the 20,000+ genes in human DNA. The project used computer programs to analyze complex genetic data, determine the sequences of the billions of chemical base pairs that make up human DNA and store the information in databases, which have been made available over the Internet to researchers in many fields.

Fig. 1.1 | Improving people's lives with computers. (Part 1 of 10.)

Name	Description
AMBER™ Alert	The AMBER (America's Missing: Broadcast Emergency Response) Alert System helps find abducted children. Law enforcement notifies TV and radio broadcasters and state transportation officials, who then broadcast alerts on TV, radio, computerized highway signs, the Internet and wireless devices. AMBER Alert partners with Facebook, whose users can “Like” AMBER Alert pages by location to receive alerts in their news feeds.

Fig. 1.1 | Improving people's lives with computers. (Part 2 of 10.)

Name	Description
World Community Grid	People worldwide can donate their unused computer processing power by installing a free secure software program that allows the World Community Grid (http://www.worldcommunitygrid.org) to harness unused capacity. This computing power, accessed over the Internet, is used in place of expensive supercomputers to conduct scientific research projects that are making a difference—providing clean water to third-world countries, fighting cancer, growing more nutritious rice for regions fighting hunger and more.

Fig. 1.1 | Improving people's lives with computers. (Part 3 of 10.)

Name	Description
Cloud computing	Cloud computing allows you to use software, hardware and information stored in the “cloud”—i.e., accessed on remote computers via the Internet and available on demand—popular examples are Dropbox, Google Drive and Microsoft OneDrive. You can increase or decrease resources incrementally to meet your needs at any given time, so cloud services can be more cost effective than purchasing expensive hardware to ensure that you have enough storage and processing power to meet peak-level needs. Using cloud-computing services shifts the burden of managing these applications from the business to the service provider, saving businesses time, effort and money. In an online chapter, you’ll use Microsoft Azure —a cloud-computing platform that allows you to develop, manage and distribute your apps in the cloud. With Microsoft Azure, your apps can store their data in the cloud so that it’s available at all times from any of your desktop computers and mobile devices. For information on Microsoft Azure’s free and paid services visit https://azure.microsoft.com .

Fig. 1.1 | Improving people’s lives with computers. (Part 4 of 10.)

Name	Description
Medical imaging	X-ray computed tomography (CT) scans, also called CAT (computerized axial tomography) scans, take X-rays of the body from hundreds of different angles. Computers are used to adjust the intensity of the X-rays, optimizing the scan for each type of tissue, then to combine all of the information to create a 3D image. MRI scanners use a technique called magnetic resonance imaging to produce internal images noninvasively.

Fig. 1.1 | Improving people's lives with computers. (Part 5 of 10.)

Name	Description
GPS	Global Positioning System (GPS) devices use a network of satellites to retrieve location-based information. Multiple satellites send time-stamped signals to the GPS device, which calculates the distance to each satellite, based on the time the signal left the satellite and the time the signal arrived. This information helps determine the device's exact location. GPS devices can provide step-by-step directions and help you locate nearby businesses (restaurants, gas stations, etc.) and points of interest. GPS is used in numerous location-based Internet services such as check-in apps to help you find your friends (e.g., Foursquare and Facebook), exercise apps such as Map My Ride+, Couch to 5K and RunKeeper that track the time, distance and average speed of your outdoor ride or jog, dating apps that help you find a match nearby and apps that dynamically update changing traffic conditions.

Fig. 1.1 | Improving people's lives with computers. (Part 6 of 10.)

Name	Description
Robots	Robots can be used for day-to-day tasks (e.g., iRobot's Roomba vacuuming robot), entertainment (e.g., robotic pets), military combat, deep sea and space exploration (e.g., NASA's Mars rover Curiosity) and more. Researchers, such as those at RoboHow (http://robohow.eu), are working to create autonomous robots that perform complex human manipulation tasks (such as cooking) and that can learn additional tasks both from the robots' own experiences and from observing humans performing other tasks.
E-mail, Instant Messaging and Video Chat	Internet-based servers support all of your online messaging. E-mail messages go through a mail server that also stores the messages. Instant Messaging (IM) and Video Chat apps, such as Facebook Messenger, WhatsApp, AIM, Skype, Yahoo! Messenger, Google Hangouts, Trillian and others, allow you to communicate with others in real time by sending your messages and live video through servers.

Fig. 1.1 | Improving people's lives with computers. (Part 7 of 10.)

Name	Description
E-commerce	This technology has exploded with companies like Amazon, eBay, Alibaba, Walmart and many others, causing a major shift away from brick-and-mortar retailers.
Internet TV	Internet TV set-top boxes (such as Apple TV, Android TV, Roku, Chromecast and TiVo) allow you to access an enormous amount of content on demand, such as games, news, movies, television shows and more, and they help ensure that the content is streamed to your TV smoothly.
Streaming music services	Streaming music services (such as Apple Music, Pandora, Spotify and more) allow you to listen to large catalogues of music over the web, create customized “radio stations” and discover new music based on your feedback.

Fig. 1.1 | Improving people’s lives with computers. (Part 8 of 10.)

Name	Description
Self-driving cars and smart homes	These are two enormous markets. Self-driving cars are under development by many technology companies and car manufacturers—they already have an impressive safety record and soon could be widely used saving lives and reducing injuries. Smart homes use computers for security, climate control, minimizing energy costs, automated lighting systems, fire detection, window control and more.

Fig. 1.1 | Improving people's lives with computers. (Part 9 of 10.)

Name	Description
Game programming	Global video-game revenues are expected to reach \$107 billion by 2017 (http://www.polygon.com/2015/4/22/8471789/worldwide-video-games-market-value-2015). The most sophisticated games can cost over \$100 million to develop, with the most expensive costing half a billion dollars (http://www.gamespot.com/gallery/20-of-the-most-expensive-games-ever-made/2900-104/). Bethesda's <i>Fallout 4</i> earned \$750 million in its first day of sales (http://fortune.com/2015/11/16/fallout4-is-quiet-best-seller/)!

Fig. 1.1 | Improving people's lives with computers. (Part 10 of 10.)

1.3 Hardware and Software

- ▶ Computers can perform calculations and make logical decisions phenomenally faster than human beings can.
- ▶ Today's personal computers can perform billions of calculations in one second—more than a human can perform in a lifetime.
- ▶ *Supercomputers* are already performing *thousands of trillions (quadrillions)* of instructions per second!
- ▶ China's National University of Defense Technology's Tianhe-2 supercomputer can perform over 33 quadrillion calculations per second (33.86 *petaflops*)!
 - *In one second about 3 million calculations for every person on the planet!*

1.3 Hardware and Software

- ▶ Computers (i.e., **hardware**) process *data* under the control of sequences of instructions called **computer programs**.
- ▶ These programs guide the computer through *actions* specified by people called **computer programmers**.
- ▶ The programs that run on a computer are referred to as **software**.

1.3.1 Moore's Law

- ▶ Every year, you probably expect to pay at least a little more for most products and services.
- ▶ The opposite has been the case in the computer and communications fields, especially with regard to the hardware supporting these technologies. For many decades, hardware costs have fallen rapidly.
- ▶ Every year or two, the capacities of computers have approximately-*doubled* inexpensively.
- ▶ This remarkable trend often is called **Moore's Law**, named for the person who identified it in the 1960s, Gordon Moore, co-founder of Intel—a leading manufacturer of the processors in today's computers and embedded systems.

1.3.1 Moore's Law

- ▶ Moore's Law and related observations apply especially to the amount of memory that computers have for programs, the amount of secondary storage (such as disk storage) they have to hold programs- and data over longer periods of time, and their processor speeds—the speeds at which they *execute* their programs (i.e., do their work).
- ▶ These increases make computers more capable, which puts greater demands on programming-language designers to innovate.
- ▶ Similar growth has occurred in the communications.



1.3.2 Computer Organization

- ▶ Regardless of differences in *physical* appearance, computers can be envisioned as divided into various **logical units** or sections (Fig. 1.2).

Logical unit	Description
Input unit	This “receiving” section obtains information (data and computer programs) from input devices and places it at the disposal of the other units for processing. Most user input is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, DVD drives, Blu-ray Disc™ drives and USB flash drives—also called “thumb drives” or “memory sticks”), receiving video from a webcam and having your computer receive information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon). Newer forms of input include position data from a GPS device, motion and orientation information from an <i>accelerometer</i> (a device that responds to up/down, left/right and forward/backward acceleration) in a smartphone or game controller (such as Microsoft® Kinect® for Xbox®, Wii™ Remote and Sony® PlayStation® Move) and voice input from devices like Amazon Echo and the forthcoming Google Home.

Logical unit	Description
Output unit	This “shipping” section takes information the computer has processed and places it on various output devices to make it available for use outside the computer. Most information that’s output from computers today is displayed on screens (including touch screens), printed on paper (“going green” discourages this), played as audio or video on PCs and media players (such as Apple’s iPods) and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as robots and “intelligent” appliances. Information is also commonly output to secondary storage devices, such as solid-state drives (SSDs), hard drives, DVD drives and USB flash drives. Popular recent forms of output are smartphone and game-controller vibration, virtual reality devices like Oculus Rift and Google Cardboard and mixed reality devices like Microsoft’s HoloLens.

Fig. 1.2 | Logical units of a computer. (Part 2 of 6.)

Logical unit	Description
Memory unit	This rapid-access, relatively low-capacity “warehouse” section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is <i>volatile</i>—it’s typically lost when the computer’s power is turned off. The memory unit is often called either memory, primary memory or RAM (Random Access Memory). Main memories on desktop and notebook computers contain as much as 128 GB of RAM, though 2 to 16 GB is most common. GB stands for gigabytes; a gigabyte is approximately one billion bytes. A byte is eight bits. A bit is either a 0 or a 1.

Fig. 1.2 | Logical units of a computer. (Part 3 of 6.)

Logical unit	Description
Arithmetic and logic unit (ALU)	This “manufacturing” section performs <i>calculations</i> , such as addition, subtraction, multiplication and division. It also contains the <i>decision</i> mechanisms that allow the computer, for example, to compare two items from the memory unit to determine whether they’re equal. In today’s systems, the ALU is implemented as part of the next logical unit, the CPU.

Fig. 1.2 | Logical units of a computer. (Part 4 of 6.)

Logical unit	Description
Central processing unit (CPU)	This “administrative” section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the memory unit to certain output devices. Many of today’s computers have multiple CPUs and, hence, can perform many operations simultaneously. A multicore processor implements multiple processors on a single integrated-circuit chip—a <i>dual-core processor</i> has two CPUs, a <i>quad-core processor</i> has four and an <i>octa-core processor</i> has eight. Today’s desktop computers have processors that can execute billions of instructions per second. Chapter 23 explores how to write apps that can take full advantage of multicore architecture.

Fig. 1.2 | Logical units of a computer. (Part 5 of 6.)

Logical unit	Description
Secondary storage unit	This is the long-term, high-capacity “warehousing” section. Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your <i>hard drive</i>) until they’re again needed, possibly hours, days, months or even years later. Information on secondary storage devices is <i>persistent</i>—it’s preserved even when the computer’s power is turned off. Secondary storage information takes much longer to access than information in primary memory, but its cost per unit is much less. Examples of secondary storage devices include solid-state drives (SSDs), hard drives, DVD drives and USB flash drives, some of which can hold over 2 TB (TB stands for terabytes; a terabyte is approximately one trillion bytes). Typical hard drives on desktop and notebook computers hold up to 2 TB, and some desktop hard drives can hold up to 6 TB.

Fig. 1.2 | Logical units of a computer. (Part 6 of 6.)

1.4 Data Hierarchy

- ▶ Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from the simplest data items (called “bits”) to richer data items, such as characters, fields, and so on.

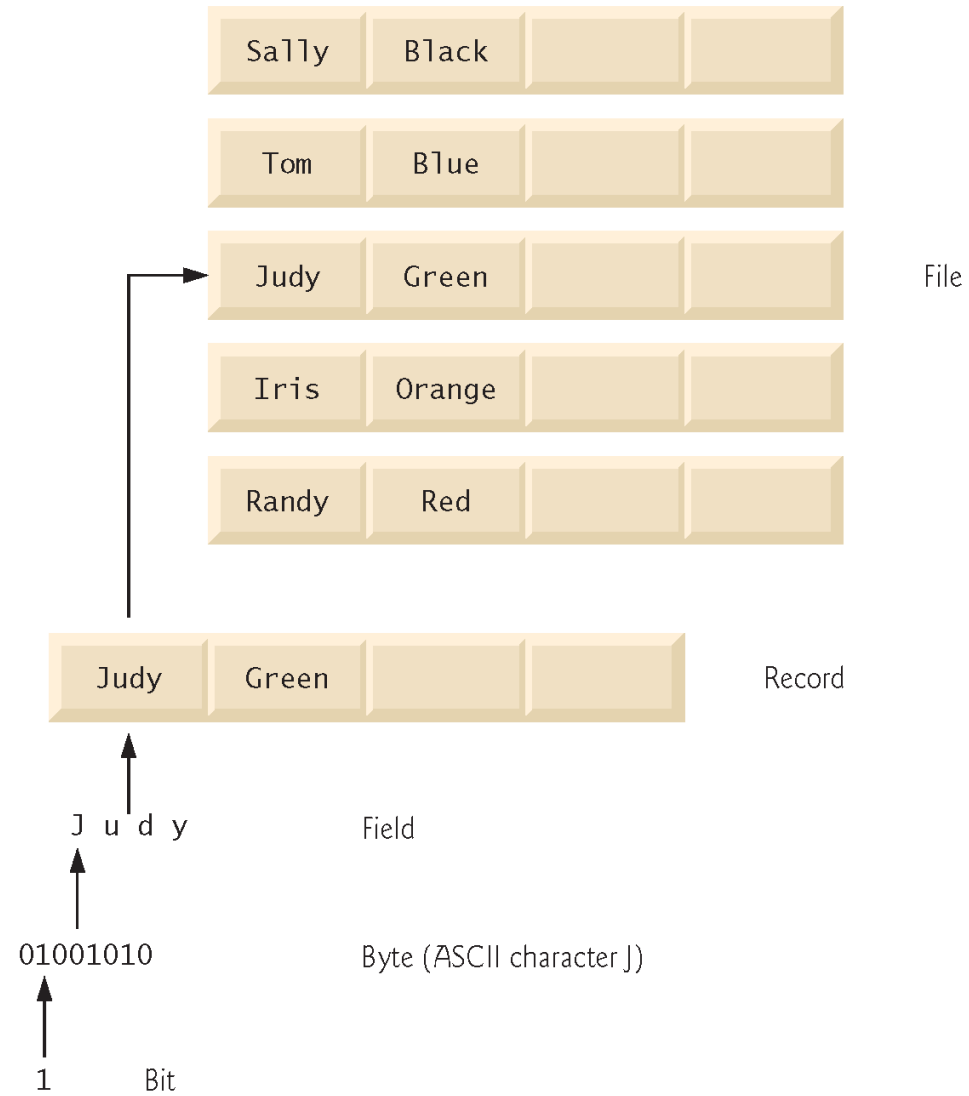


Fig. 1.3 | Data hierarchy.

1.4 Data Hierarchy

Bits

- ▶ The smallest data item in a computer can assume the value 0 or the value 1.
- ▶ Such a data item is called a **bit** (short for “binary digit”—a digit that can assume either of two values).
- ▶ It’s remarkable that the impressive functions performed by computers involve only the simplest manipulations of 0s and 1s—*examining a bit’s value, setting a bit’s value and reversing a bit’s value* (from 1 to 0 or from 0 to 1).

1.4 Data Hierarchy

Characters

- ▶ We prefer to work with *decimal digits* (0–9), *uppercase letters* (A–Z), *lowercase letters* (a–z), and *special symbols* (e.g., \$, @, %, &, *, (,), –, +, ", :, ? and /).
- ▶ Digits, letters and special symbols are known as characters. The computer's character set is the set of all the **characters** used to write programs and represent data items on that device.
- ▶ Computers process only 1s and 0s, so every character is represented as a pattern of 1s and 0s.
- ▶ The **Unicode** character set contains characters for many of the world's languages.

1.4 Data Hierarchy

- ▶ C# supports several character sets, including 16-bit Unicode[®] characters that are composed of two bytes—each byte is composed of eight bits.
- ▶ See Appendix B for more information on the [ASCII \(American Standard Code for Information Interchange\)](#) character set—the popular *subset* of Unicode that represents uppercase and lowercase letters in the English alphabet, digits and some common *special characters*.

1.4 Data Hierarchy

Fields

- ▶ Just as characters are composed of bits, **fields** are composed of characters or bytes.
- ▶ A field is a group of characters or bytes that conveys meaning.
- ▶ For example, a field consisting of uppercase and lowercase letters could be used to represent a person's name, and a field consisting of decimal digits could represent a person's age.

1.4 Data Hierarchy

Records

- ▶ Several related fields can be used to compose a **record**.
- ▶ In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses):
 - Employee identification number (a whole number)
 - Name (a string of characters)
 - Address (a string of characters)
 - Hourly pay rate (a number with a decimal point)
 - Year-to-date earnings (a number with a decimal point)
 - Amount of taxes withheld (a number with a decimal point)
- ▶ Thus, a record is a group of related fields.
- ▶ In the preceding example, all the fields belong to the same employee.

1.4 Data Hierarchy

Files

- ▶ A **file** is a group of related records.
- ▶ More generally, a file contains arbitrary data in arbitrary formats.
- ▶ In some operating systems, a file is viewed simply as a *sequence of bytes*—any organization of the bytes in a file, such as organizing the data into records, is a view created by the programmer.

1.4 Data Hierarchy

Database

- ▶ A **database** is a collection of data that's organized for easy access and manipulation.
- ▶ The most popular database model is the *relational database* in which data is stored in simple *tables*.
- ▶ A table includes *records* composed of *fields*.
 - For example, a table of students might include first name, last name, major, year, student ID number and grade point average fields.
 - The data for each student is a record, and the individual pieces of information in each record are the fields.
- ▶ You can *search*, *sort* and otherwise manipulate the data based on its relationship to multiple tables or databases.

1.4 Data Hierarchy

- ▶ The amount of data being produced worldwide is enormous and growing quickly.
- ▶ Approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily, and as of October 2015 90% of the world's data was created in just the prior 12 months.
- ▶ Figure 1.4 shows some common byte measurements.
- ▶ **Big data** applications deal with massive amounts of data and this field is growing quickly, creating lots of opportunity for software developers.

Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024) bytes exactly
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000) bytes
1 gigabyte (GB)	1024 mega- bytes	10^9 (1,000,000,000) bytes
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000) bytes
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000) bytes
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000) bytes
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000) bytes

Fig. 1.4 | Byte measurements.



1.5 Machine Languages, Assembly Languages and High-Level Languages

- ▶ Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation* steps.
- ▶ Any computer can directly understand only its own **machine language** (also called *machine code*), defined by its hard-ware architecture.
- ▶ Machine languages generally consist of numbers (ultimately reduced to 1s and 0s). Such languages are cum-bersome for humans.



1.5 Machine Languages, Assembly Languages and High-Level Languages

- ▶ English-like *abbreviations* formed the basis of **assembly languages**.
- ▶ *Translator programs* called **assemblers** were developed to convert assembly-language programs to machine language.
- ▶ Although assembly-language code is clearer to humans, it's incomprehensible to computers until translated to machine language.



1.5 Machine Languages, Assembly Languages and High-Level Languages

- ▶ **High-level languages** were developed in which single statements could be written to accomplish substantial tasks.
- ▶ High-level languages, such as C#, Visual Basic, C, C++, Java and Swift, allow you to write instructions that look more like everyday English and contain commonly used mathematical notations.
- ▶ Translator programs called **compilers** convert high-level language programs into machine language.
- ▶ **Interpreter** programs execute high-level language programs directly (without the need for compilation), although more slowly than compiled programs.
- ▶ **Scripting languages** such as the popular web languages JavaScript and PHP are processed by interpreters.



Performance Tip 1.1

Interpreters have an advantage over compilers in Internet scripting. An interpreted program can begin executing as soon as it's downloaded to the client's machine, without needing to be compiled before it can execute. On the downside, interpreted scripts generally run slower and consume more memory than compiled code. With a technique called JIT (just-in-time) compilation, interpreted languages can often run almost as fast as compiled ones.

1.6 Object Technology

- ▶ C# is an object-oriented programming language.
- ▶ **Objects**, or more precisely—as we'll see in Chapter 4—the **classes** objects come from, are essentially *reusable* software components.
- ▶ Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating-, moving and communicating).
- ▶ A modular, object-oriented design-and-implementation-approach can make software-development groups much more productive.

1.6 Object Technology

- ▶ ***The Automobile as an Object***
- ▶ Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this?
- ▶ Someone has to *design* it.
- ▶ A car typically begins as engineering drawings, similar to the *blueprints* that describe the design of a house.
- ▶ These drawings include the design for an accelerator pedal.
- ▶ The pedal *hides* from the driver the complex mechanisms that actually make the car go faster, just as the brake pedal hides the mechanisms that slow the car, and the steering wheel *hides* the mechanisms that turn the car.

1.6 Object Technology

- ▶ ***The Automobile as an Object***
- ▶ This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.
- ▶ Before you can drive a car, it must be *built* from the engineering drawings that describe it.
- ▶ A completed car has an *actual* accelerator pedal to make the car go faster, but even that's not enough—the car won't accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.



1.6 Object Technology

- ▶ ***Methods and Classes***
- ▶ Performing a task in a program requires a **method**.
- ▶ The method houses the program statements that actually perform the task.
- ▶ It *hides* these statements from its user, just as a car's accelerator pedal hides from the driver the mechanisms of making the car go faster.
- ▶ In C#, we create a program unit called a class to house the set of methods that perform the class's tasks.
- ▶ A class is similar in concept to a car's engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

1.6 Object Technology

- ▶ ***Making Objects from Classes***
- ▶ Just as someone has to *build a car* from its engineering drawings before you can actually drive a car, you must *build an object* from a class before a program can perform the tasks that the class's methods define.
- ▶ The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

1.6 Object Technology

- ▶ ***Reuse***
- ▶ Just as a car's engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects.
- ▶ Reuse of existing classes when building new classes and programs saves time and effort.
- ▶ Reuse also helps you build more reliable and effective systems, because existing classes and components often have gone through extensive *testing* (to locate problems), *debugging* (to correct those problems) and *performance tuning*.

1.6 Object Technology

- ▶ ***Messages and Method Calls***
- ▶ When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster.
- ▶ Similarly, you *send messages to an object*.
- ▶ Each message is implemented as a **method call** that tells a method of the object to perform its task.

1.6 Object Technology

- ▶ *Attributes and Instance Variables*
- ▶ A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total miles driven (i.e., its odometer reading).
- ▶ An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class.
- ▶ Attributes are specified by the class's **instance variables**.

1.6 Object Technology

- ▶ ***Properties, get Accessors and set Accessors***
- ▶ Attributes are not necessarily accessible directly.
- ▶ The car manufacturer does not want drivers to take apart the car's engine to observe the amount of gas in its tank.
- ▶ Instead, the driver can check the fuel gauge on the dashboard.
- ▶ Similarly, you do not need to have access to an object's instance variables in order to use them.
- ▶ You should use the **properties** of an object.
 - Properties contain **get accessors** for reading the values of variables, and **set accessors** for storing values into them.

1.6 Object Technology

- ▶ **Encapsulation**
- ▶ Classes **encapsulate** (i.e., wrap) attributes and methods into objects created from those classes—an object's attributes and methods are intimately related.
- ▶ Objects may communicate with one another, but they're normally not allowed to know how other objects are implemented—implementation details are *hidden* within the objects themselves.
- ▶ This **information hiding**, as we'll see, is crucial to good software engineering.

1.6 Object Technology

- ▶ ***Inheritance***
- ▶ A new class of objects can be created quickly and conveniently by **inheritance**—the new class absorbs the characteristics of an existing class, possibly customizing them and adding unique characteristics of its own.
- ▶ In our car analogy, an object of class “convertible” certainly *is an* object of the more *general* class “automobile,” but more *specifically*, the roof can be raised or lowered.

1.6 Object Technology

- ▶ ***Object-Oriented Analysis and Design (OOAD)***
- ▶ To create the best solutions, you should follow a detailed **analysis** process for determining your project's **requirements** (i.e., defining *what* the system is supposed to do) and developing a **design** that satisfies them (i.e., deciding *how* the system should do it).
- ▶ Ideally, you'd go through this process and carefully review the design (and have your design reviewed by other software professionals) before writing any code.
- ▶ If this process involves analyzing and designing your system from an object-oriented point of view, it's called an **object-oriented analysis and design (OOAD) process**.

1.6 Object Technology

- ▶ ***The UML (Unified Modeling Language)***
- ▶ Although many different OOAD processes exist, a single graphical language for communicating the results of *any* OOAD process has come into wide use.
- ▶ This language, known as the Unified Modeling Language (UML), is now the most widely used graphical scheme for modeling object-oriented systems.

1.7 Internet and World Wide Web

- ▶ In the late 1960s, ARPA—the Advanced Research Projects Agency of the Department of Defense—rolled out plans to network the main computer systems of approximately a dozen ARPA-funded universities and research institutions.
- ▶ ARPA implemented what quickly became known as the ARPAnet, the precursor of today's **Internet**.
- ▶ Its main benefit proved to be the capability for quick and easy communication via e-mail.
- ▶ This is true even on today's Internet, with e-mail, instant messaging, file transfer and social media such as Facebook and Twitter, enabling billions of people worldwide to communicate quickly and easily.

1.7 Internet and World Wide Web

- ▶ The protocol (set of rules) for communicating over the ARPAnet became known as the **Transmission Control Protocol (TCP)**.
- ▶ TCP ensured that messages, consisting of sequentially numbered pieces called *packets*, were properly routed from sender to receiver, arrived intact and were assembled in the correct order.

1.7 Internet and World Wide Web

The Internet: A Network of Networks

- ▶ In parallel with the early evolution of the Internet, organizations worldwide were implementing their own networks for both intraorganization (that is, within an organization) and interorganization (that is, between organizations) communication.
- ▶ One challenge was to enable these different networks to communicate with each other.
- ▶ The Internet Protocol (IP) created a true “network of networks,” the current architecture of the Internet.
- ▶ The combined set of protocols is now called **TCP/IP**.

1.7 Internet and World Wide Web

- ▶ Businesses rapidly realized that by using the Internet, they could improve their operations and offer new and better services to their clients.
- ▶ This generated fierce competition among communications carriers and hardware and software suppliers to meet the increased infrastructure demand.
- ▶ As a result, **bandwidth**—the information-carrying capacity of communications lines—on the Internet has increased tremendously, while hardware costs have plummeted.



1.7 Internet and World Wide Web

- ▶ The **World Wide Web** (simply called “the web”) is a collection of hardware and software associated with the Internet that allows computer users to locate and view multimedia-based documents (documents with various combinations of text, graphics, animations, audios and videos) on almost any subject.

1.7 Internet and World Wide Web

- ▶ In 1989, Tim Berners-Lee of CERN (the European Organization for Nuclear Research) began to develop a technology for sharing information via “hyperlinked” text documents.
- ▶ Berners-Lee called his invention the **HyperText Markup Language (HTML)**.
- ▶ He also wrote communication protocols such as **HyperText Transfer Protocol (HTTP)** to form the backbone of his new hypertext information system, which he referred to as the World Wide Web.

1.7 Internet and World Wide Web

- ▶ In 1994, Berners-Lee founded the **World Wide Web Consortium (W3C)**, devoted to developing web technologies.
- ▶ One of the W3C's goals is to make the web accessible to everyone regardless of disabilities, language or culture.
- ▶ You'll use C# and other Microsoft technologies to build web-based apps.



1.8 C#

- ▶ In 2000, Microsoft announced the **C#** programming language.
- ▶ C# has roots in the C, C++ and Java programming languages.



1.8.1 Object-Oriented Programming

- ▶ C# is *object oriented*—we've discussed the basics of object technology and we present a rich treatment of object-oriented programming throughout the book.
- ▶ C# has access to the powerful **.NET Framework Class Library**—a vast collection of prebuilt classes that enable- you to develop apps quickly (Fig. 1.5).

Some key capabilities in the .NET Framework Class Library

Database

Building web apps

Graphics

Input/output

Computer networking

Permissions

Mobile

String processing

Debugging

Multithreading

File processing

Security

Web communication

Graphical user interface

Data structures

Universal Windows Platform GUI

Fig. 1.5 | Some key capabilities in the .NET Framework Class Library.

1.8.2 Event-Driven Programming

- ▶ C# graphical user interfaces (GUIs) are **event driven**.
- ▶ You can write programs that respond to user-initiated **events** such as mouse clicks, keystrokes, timer expirations and *touches* and *finger swipes*—gestures that are widely used on smartphones and tablets.

1.8.3 Visual Programming

- ▶ Microsoft's Visual Studio enables you to use C# as a *visual programming language*.
- ▶ You'll use Visual Studio to conveniently drag and drop predefined GUI objects like *buttons* and *textboxes* into place on your screen, and label and resize them.
- ▶ Visual Studio will write much of the GUI code for you.

18.4 Generic and Functional Programming

- ▶ ***Generic Programming***
- ▶ It's common to write a program that processes a collection of things.
- ▶ Historically, you had to program separately to handle each type of collection.
- ▶ With generic programming, you write code that handles a collection “in the general” and C# handles the specifics for each different type of collection.

1.8.4 Generic and Functional Programming

- ▶ ***Functional Programming***
- ▶ With *functional programming*, you specify *what* you want to accomplish in a task, but *not how* to accomplish it.
- ▶ For example, with Microsoft's LINQ you can say, "Here's a collection of numbers, give me the sum of its elements."
- ▶ You do *not* need to specify the mechanics of walking through the elements and adding them into a running total one at a time—LINQ handles all that for you.
- ▶ Functional programming speeds application development and reduces errors.

1.8.5 An International Standard

- ▶ C# has been standardized through ECMA International:
 - <http://www.ecma-international.org>
- ▶ At the time of this writing, the C# standard document—ECMA-334—was still being updated for C# 6.
- ▶ For information on ECMA-334, visit
 - <http://www.ecma-international.org/publications/standards/Ecma-334.htm>
- ▶ Visit the Microsoft download center to find the latest version of Microsoft's C# 6 specification, other documentation and software downloads.



1.8.6 C# on Non-Windows Platforms

- ▶ Though C# was originally developed by Microsoft for the Windows platform, the language can be used on other platforms via the **Mono Project** and **.NET Core**—both are managed by the .NET Foundation
 - <http://www.dotnetfoundation.org/>



1.8.7 Internet and Web Programming

- ▶ Today's apps can be written with the aim of communicating among the world's computers.
- ▶ This is the focus of Microsoft's .NET strategy.



1.8.7 Internet and Web Programming

- ▶ Today's apps can be written with the aim of communicating among the world's computers.
- ▶ This is the focus of Microsoft's .NET strategy.



1.8.8. Asynchronous Programming with `async` and `await`

- ▶ In most programming today, each task in a program must finish executing before the next task can begin.
- ▶ This is called *synchronous programming* and is the style we use for most of this book.
- ▶ C# also allows *asynchronous programming* in which multiple tasks can be performed at the *same* time.
- ▶ Asynchronous programming can help you make your apps more responsive to user interactions, such as mouse clicks and keystrokes, among many other uses.



1.8.8. Asynchronous Programming with `async` and `await`

- ▶ Asynchronous programming in early versions of Visual C# was difficult and error prone.
- ▶ C#'s `async` and `await` capabilities simplify asynchronous programming by enabling the compiler to hide much of the associated complexity from the developer.



1.8.9 Other Key Programming Languages

- ▶ Figure 1.6 provides brief comments on several popular programming languages.

Programming language	Description
Ada	Ada, based on Pascal, was developed under the sponsorship of the U.S. Department of Defense (DOD) during the 1970s and early 1980s. The DOD wanted a single language that would fill most of its needs. The Pascal-based language was named after Lady Ada Lovelace, daughter of the poet Lord Byron. She's credited with writing the world's first computer program in the early 1800s (for the Analytical Engine mechanical computing device designed by Charles Babbage). Ada also supports object-oriented programming.
Basic	Basic was developed in the 1960s at Dartmouth College to familiarize novices with programming techniques. Many of its latest versions are object oriented.

Fig. 1.6 | Some other programming languages. (Part 1 of 9.)

Programming language	Description
C	C was developed in the early 1970s by Dennis Ritchie at Bell Laboratories. It initially became widely known as the UNIX operating system's development language. Today, most code for general-purpose operating systems is written in C or C++.
C++	C++, which is based on C, was developed by Bjarne Stroustrup in the early 1980s at Bell Laboratories. C++ provides several features that “spruce up” the C language, but more important, it provides capabilities for object-oriented programming.

Fig. 1.6 | Some other programming languages. (Part 2 of 9.)

Programming language	Description
COBOL	COBOL (COmmon Business Oriented Language) was developed in the late 1950s by computer manufacturers, the U.S. government and industrial computer users, based on a language developed by Grace Hopper, a career U.S. Navy officer and computer scientist. COBOL is still widely used for commercial applications that require precise and efficient manipulation of large amounts of data. Its latest version supports object-oriented programming.
Fortran	Fortran (FORmula TRANslator) was developed by IBM Corporation in the mid-1950s to be used for scientific and engineering applications that require complex mathematical computations. It's still widely used, and its latest versions support object-oriented programming.

Fig. 1.6 | Some other programming languages. (Part 3 of 9.)

Programming language	Description
Java	Sun Microsystems in 1991 funded an internal corporate research project led by James Gosling, which resulted in the C++-based object-oriented programming language called Java. A key goal of Java is to enable developers to write programs that will run on a great variety of computer systems and computer-controlled devices. This is sometimes called “write once, run anywhere.” Java is used to develop large-scale enterprise applications, to enhance the functionality of web servers (the computers that provide the content we see in our web browsers), to provide applications for consumer devices (e.g., smartphones, tablets, television set-top boxes, appliances, automobiles and more) and for many other purposes. Java is also the key language for developing Android smartphone and tablet apps.

Fig. 1.6 | Some other programming languages. (Part 4 of 9.)

Programming language	Description
Objective-C	Objective-C is an object-oriented language based on C. It was developed in the early 1980s and later acquired by NeXT, which in turn was acquired by Apple. It became the key programming language for the OS X operating system and all iOS-powered devices (such as iPods, iPhones and iPads).
JavaScript	JavaScript is the most widely used scripting language. It's primarily used to add programmability to web pages—for example, animations and interactivity with the user. All major web browsers support it.

Fig. 1.6 | Some other programming languages. (Part 5 of 9.)

Programming language	Description
Pascal	Research in the 1960s resulted in <i>structured programming</i> —a disciplined approach to writing programs that are clearer, easier to test and debug and easier to modify than programs produced with previous techniques. The Pascal language, developed by Professor Niklaus Wirth in 1971, grew out of this research. It was popular for teaching structured programming for several decades.
PHP	PHP is an object-oriented, <i>open-source</i> “scripting” language supported by a community of developers and used by numerous websites. PHP is platform independent—implementations exist for all major UNIX, Linux, Mac and Windows operating systems.

Fig. 1.6 | Some other programming languages. (Part 6 of 9.)

Programming language	Description
Python	Python, another object-oriented scripting language, was released publicly in 1991. Developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam (CWI), Python draws heavily from Modula-3—a systems programming language. Python is “extensible”—it can be extended through classes and programming interfaces.
Swift	Swift, which was introduced in 2014, is Apple’s programming language of the future for developing iOS and OS X applications (apps). Swift is a contemporary language that includes popular programming-language features from languages such as Objective-C, Java, C#, Ruby, Python and others. In 2015, Apple released Swift 2 with new and updated features. According to the Tiobe Index, Swift has already become one of the most popular programming languages. Swift is now <i>open source</i> , so it can be used on non-Apple platforms as well.

Fig. 1.6 | Some other programming languages. (Part 7 of 9.)

Programming language	Description
Ruby on Rails	Ruby—created in the mid-1990s by Yukihiro Matsumoto—is an open-source, object-oriented programming language with a simple syntax that’s similar to Python. Ruby on Rails combines the scripting language Ruby with the Rails web-application framework developed by the company 37Signals. Their book, <i>Getting Real</i> (free at http://gettingreal.37signals.com/toc.php), is a must-read for web developers. Many Ruby on Rails developers have reported productivity gains over other languages when developing database-intensive web applications.

Fig. 1.6 | Some other programming languages. (Part 8 of 9.)

Programming language	Description
Scala	Scala (http://www.scala-lang.org/what-is-scala.html)—short for “scalable language”—was designed by Martin Odersky, a professor at École Polytechnique Fédérale de Lausanne (EPFL) in Switzerland. Released in 2003, Scala uses both the object-oriented programming and functional programming paradigms and is designed to integrate with Java. Programming in Scala can reduce the amount of code in your applications significantly.
Visual Basic	Microsoft’s Visual Basic language was introduced in the early 1990s to simplify the development of Microsoft Windows applications. Its latest versions support object-oriented programming.

Fig. 1.6 | Some other programming languages. (Part 9 of 9.)

1.9 Microsoft's .NET

- ▶ In 2000, Microsoft announced its **.NET initiative** (www.microsoft.com/net), a broad vision for using the Internet and the web in the development, engineering, distribution and use of software.
- ▶ Rather than forcing you to use a single programming language, .NET permits you to create apps in *any* .NET-compatible language (such as C#, Visual Basic, Visual C++ and others).

1.9.1 .NET Framework

- ▶ The **.NET Framework Class Library** provides many capabilities that you'll use to build substantial C# apps quickly and easily.
- ▶ It contains *thousands* of valuable *prebuilt* classes that have been tested and tuned to maximize performance.
- ▶ You should *re-use* the .NET Framework classes whenever possible to speed up the software-development process, while enhancing the quality and performance of the software you develop.

1.9.2 Common Language Runtime

- ▶ The **Common Language Runtime (CLR)** executes .NET programs and provides functionality to make them easier to develop and debug.
- ▶ The CLR is a **virtual machine (VM)**—software that manages the execution of programs and hides from them the underlying operating system and hardware.
- ▶ The source code for programs that are executed and managed by the CLR is called *managed code*.

1.9.2 Common Language Runtime

- ▶ The CLR provides various services to managed code
 - integrating software components written in different .NET languages,
 - error handling between such components,
 - enhanced security,
 - automatic memory management and more.
- ▶ Unmanaged-code programs do not have access to the CLR's services, which makes unmanaged code more difficult to write.

1.9.2 Common Language Runtime

- ▶ Managed code is compiled into machine-specific instructions in the following steps:
 - First, the code is compiled into **Microsoft Intermediate Language (MSIL)**.
Code converted into MSIL from other languages and sources can be woven together by the CLR—this allows programmers to work in their preferred .NET programming language.
The MSIL for an app's components is placed into the app's *executable file*—the file that causes the computer to perform the app's tasks.
 - When the app executes, another compiler (known as the **just-in-time compiler** or **JIT compiler**) in the CLR translates the MSIL in the executable file into machine-language code (for a particular platform).
 - The machine-language code executes on that platform.

1.9.3 Platform Independence

- ▶ If the .NET Framework exists and is installed for a platform, that platform can run *any* .NET program.
- ▶ The ability of a program to run without modification across multiple platforms is known as **platform independence**.
 - Code written once can be used on another type of computer without modification, saving time and money.

1.9.4 Language Interoperability

- ▶ The .NET Framework provides a high level of **language interoperability**.
- ▶ Because software components written in different .NET languages (such as C# and Visual Basic) are all compiled into MSIL, the components can be combined to create a single unified program.
 - Thus, MSIL allows the .NET Framework to be **language independent**.

1.9.4 Language Interoperabilit

- ▶ The .NET Framework Class Library can be used by any .NET language. The latest release of .NET includes .NET 4.6 and .NET Core:
 - .NET 4.6 introduces many improvements and new features, including ASP.NET 5 for web-based applications, improved support for today's high-resolution 4K screens and more.
 - .NET Core is the cross-platform subset of .NET for Windows, Linux, OS X and FreeBSD.

1.10 Microsoft's Windows Operating System

- ▶ Microsoft's Windows is the most widely personal-computer, desktop operating system worldwide.
- ▶ **Operating systems** are software systems that make using computers more convenient for users, developers and system administrators.
- ▶ They provide *services* that allow each app to execute safely, efficiently and *concurrently* (i.e., in parallel) with other apps.
- ▶ Other popular desktop operating systems include Mac OS X and Linux.
- ▶ Figure 1.7 presents the evolution of the Windows operating system.

Version	Description
Windows in the 1990s	In the mid-1980s, Microsoft developed the Windows operating system based on a graphical user interface with buttons, textboxes, menus and other graphical elements. The various versions released throughout the 1990s were intended for personal computing. Microsoft entered the corporate operating systems market with the 1993 release of <i>Windows NT</i> .
Windows XP and Windows Vista	<i>Windows XP</i> was released in 2001 and combined Microsoft's corporate and consumer operating-system lines. At the time of this writing, it still holds more than 10% of the operating-systems market (https://www.netmarket-share.com/operating-system-market-share.aspx). <i>Windows Vista</i> , released in 2007, offered the attractive new Aero user interface, many powerful enhancements and new apps and enhanced security. But Vista never caught on.

Fig. 1.7 | The evolution of the Windows operating system. (Part 1 of 4.)

Version	Description
Windows 7	<i>Windows 7</i> is currently the world's most widely used desktop operating system with over 47% of the operating-systems market (https://www.netmarketshare.com/operating-system-market-share.aspx). Windows added enhancements to the Aero user interface, faster startup times, further refinement of Vista's security features, touch-screen with multitouch support, and more.
Windows 8 for Desktops and Tablets	Windows 8, released in 2012, provided a similar platform (the underlying system on which apps run) and <i>user experience</i> across a wide range of devices including personal computers, smartphones, tablets <i>and</i> the Xbox Live online game service. Its new look-and-feel featured a Start screen with <i>tiles</i> representing each app, similar to that of <i>Windows Phone</i> —Microsoft's smartphone operating system. Windows 8 featured <i>multitouch</i> support for <i>touchpads</i> and <i>touchscreen</i> devices, enhanced security features and more.

Fig. 1.7 | The evolution of the Windows operating system. (Part 2 of 4.)

Version	Description
Windows 8 UI (User Interface)	Windows 8 UI (previously called “Metro”) introduced a clean look-and-feel with minimal distractions to the user. Windows 8 apps featured a <i>chromeless window</i> with no borders, title bars and menus. These elements were <i>hidden</i> , allowing apps to fill the <i>entire</i> screen—particularly helpful on smaller screens such as tablets and smartphones. The interface elements were displayed in the <i>app bar</i> when the user <i>swiped</i> the top or bottom of the screen by holding down the mouse button, moving the mouse in the swipe direction and releasing the mouse button; or using a <i>finger swipe</i> on a touchscreen device.

Fig. 1.7 | The evolution of the Windows operating system. (Part 3 of 4.)

Version	Description
Windows 10 and the Universal Windows Platform	Windows 10, released in 2015, is the current version of Windows and currently holds a 15% (and growing) share of the operating-systems market (https://www.netmarketshare.com/operating-system-market-share.aspx). In addition to many user-interface and other updates, Windows 10 introduced the Universal Windows Platform (UWP) , which is designed to provide a common platform (the underlying system on which apps run) and user experience across all Windows devices including personal computers, smartphones, tablets, Xbox and even Microsoft's new HoloLens augmented reality holographic headset—all using nearly identical code.

Fig. 1.7 | The evolution of the Windows operating system. (Part 4 of 4.)



1.10 Microsoft's Windows Operating System

- ▶ ***Windows Store***
- ▶ You can sell apps or offer them for free in the Windows Store. See the App Developer Agreement for more information:
 - <https://msdn.microsoft.com/en-us/library/windows/apps/hh694058.aspx>
- ▶ Several business models for monetizing your app:
 - Charge full price for your app before download, with prices starting at \$1.49.
 - Offer a time-limited trial or feature-limited trial that allows users to try the app before purchasing the full version, sell virtual goods (such as additional app features) using in-app purchases and more.
 - <https://msdn.microsoft.com/windows/uwp/monetize/index>

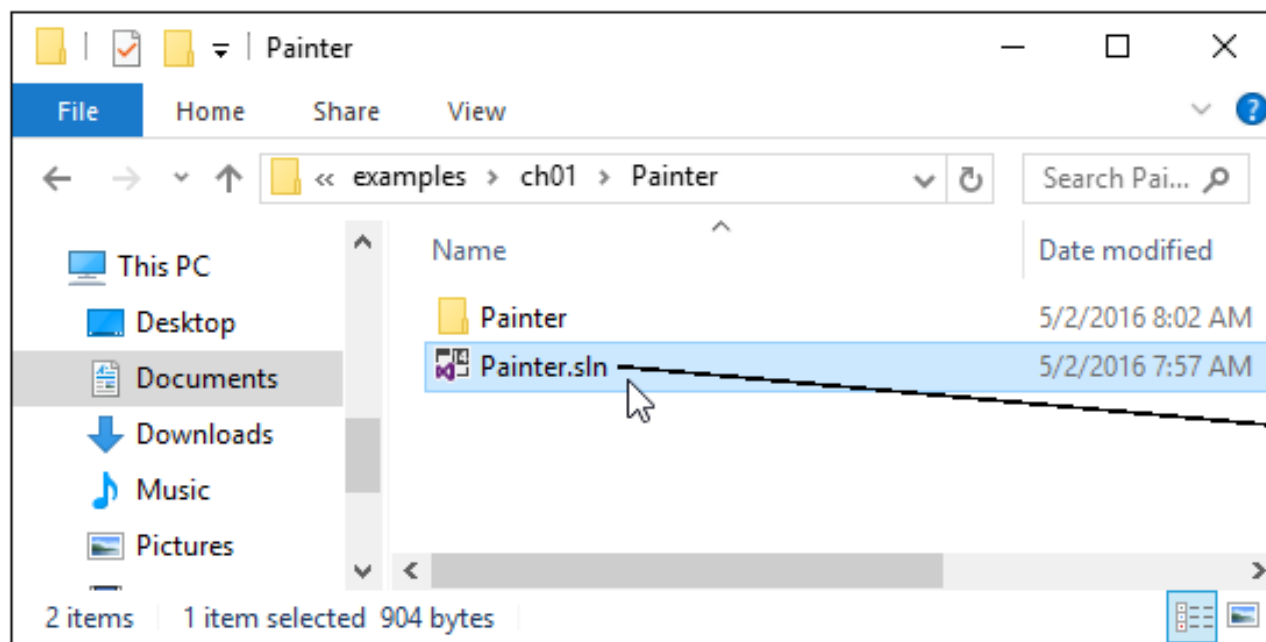


1.11 Visual Studio Integrated Development Environment

- ▶ C# programs can be created using Microsoft's Visual Studio—a collection of software tools called an **Integrated Development Environment (IDE)**.
- ▶ The **Visual Studio Community** edition IDE enables you to *write, run, test* and *debug* C# programs quickly and conveniently.
- ▶ It also supports Microsoft's Visual Basic, Visual C++ and F# programming languages and more.



1.12 Painter Test-Drive in Visual Studio Community



Double click **Painter.sln** to open the project in Visual Studio

Fig. 1.8 | Contents of C:\examples\ch01\Painter.

Press the **Start** button to begin executing the **Painter** app

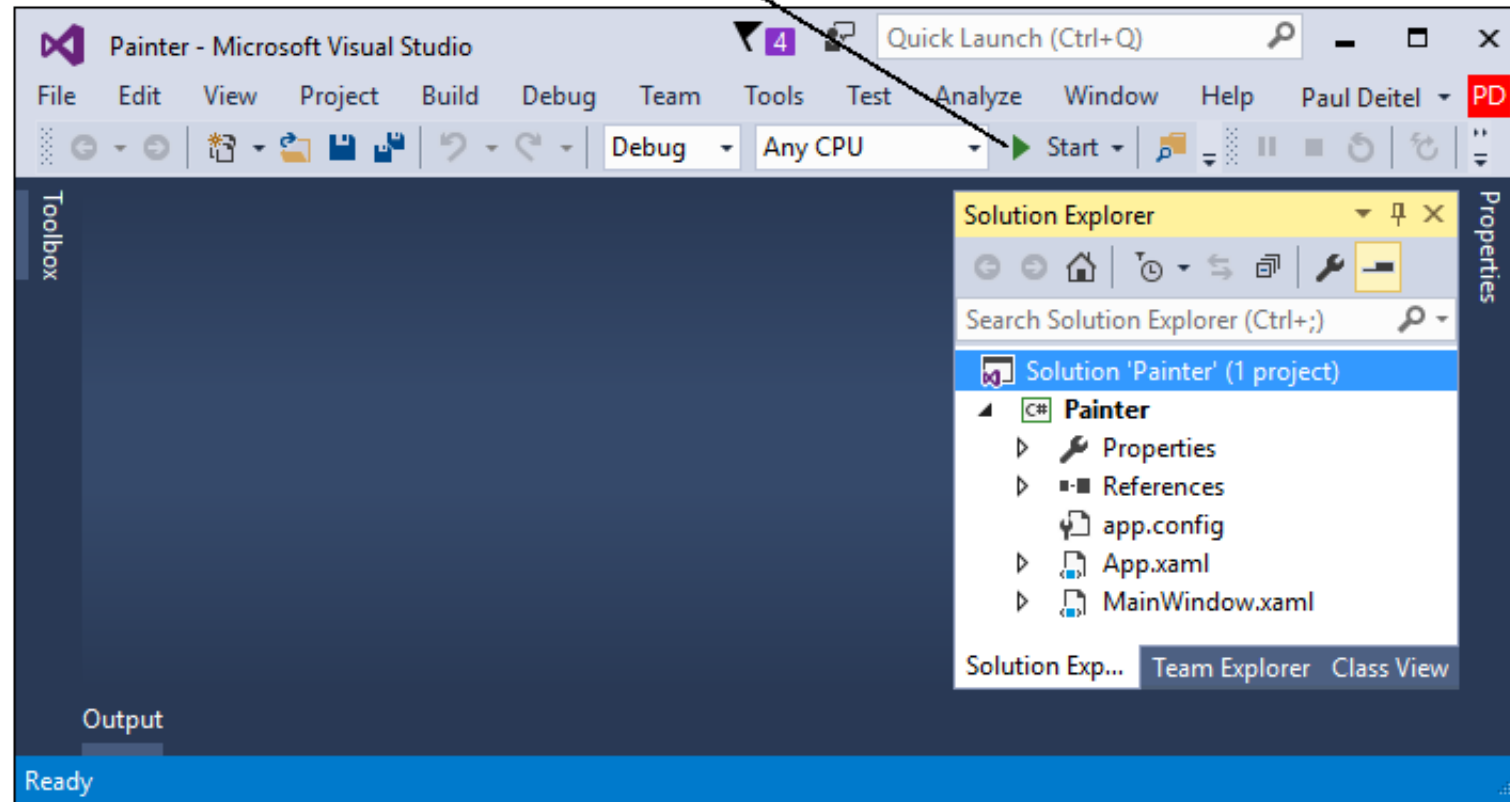


Fig. 1.9 | Running the **Painter** app.

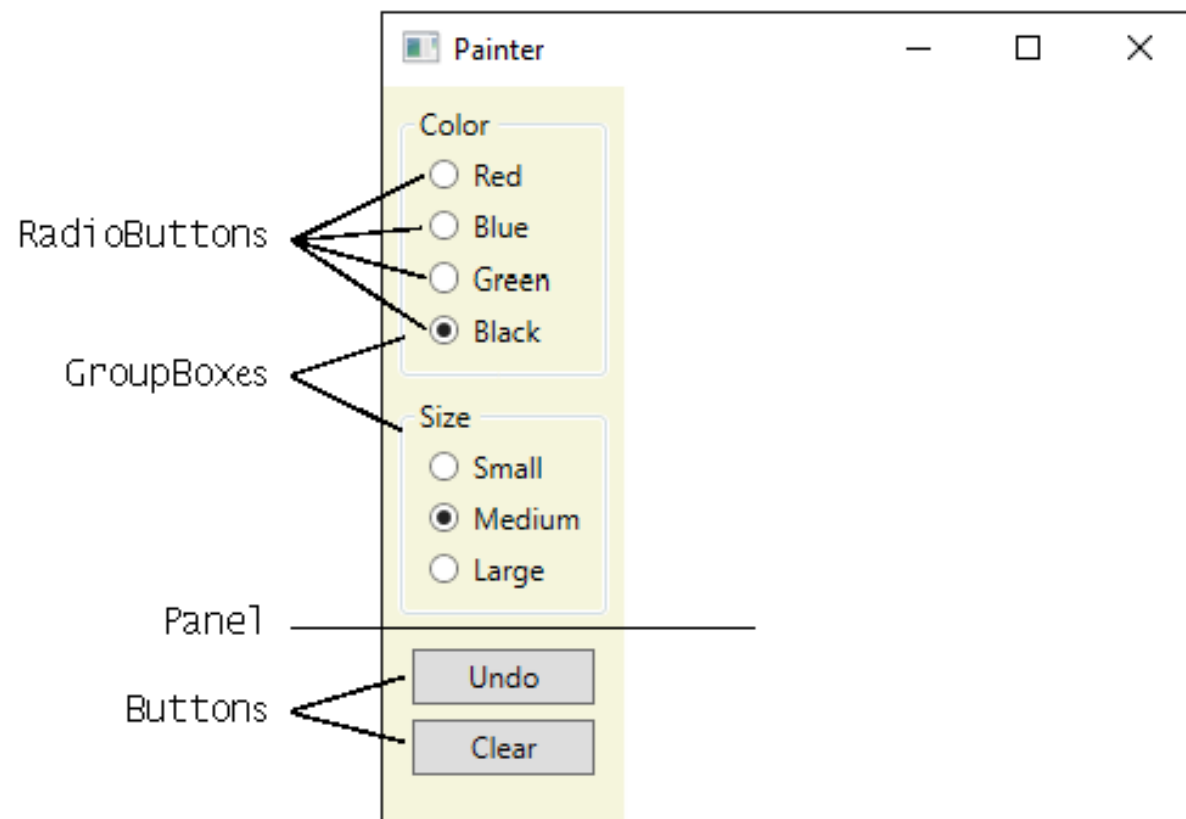


Fig. 1.10 | **Painter** app running in Windows 10.

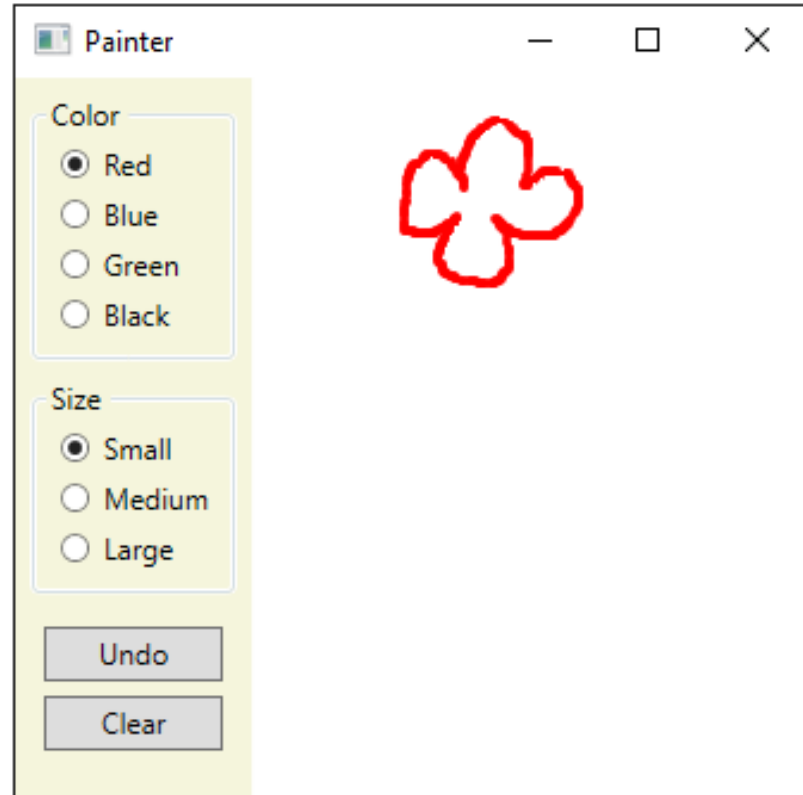


Fig. 1.11 | Drawing flower petals with a small red brush.

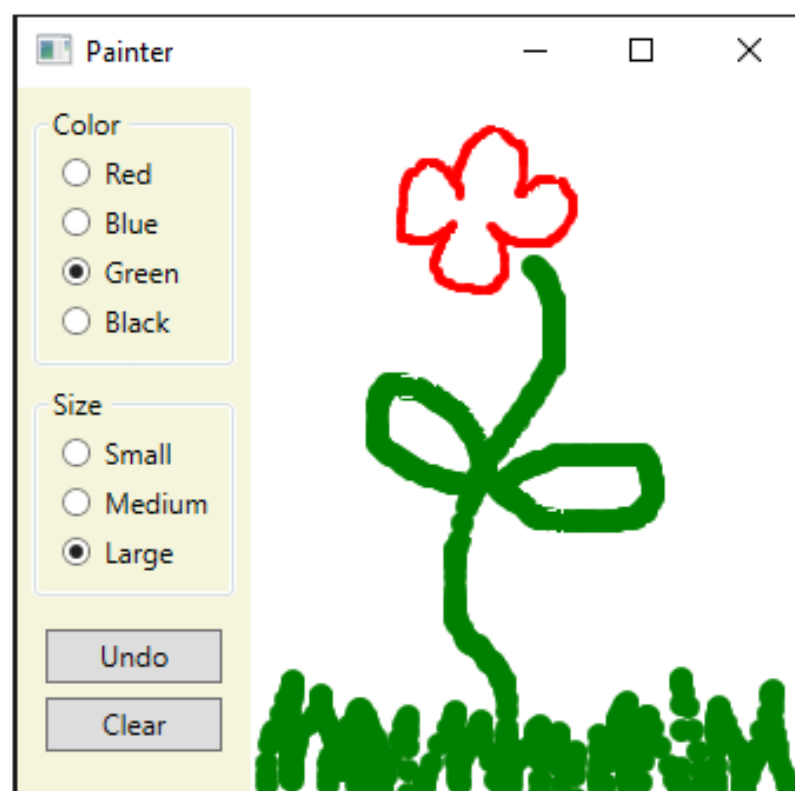


Fig. 1.12 | Drawing the flower stem and grass with a large green brush.

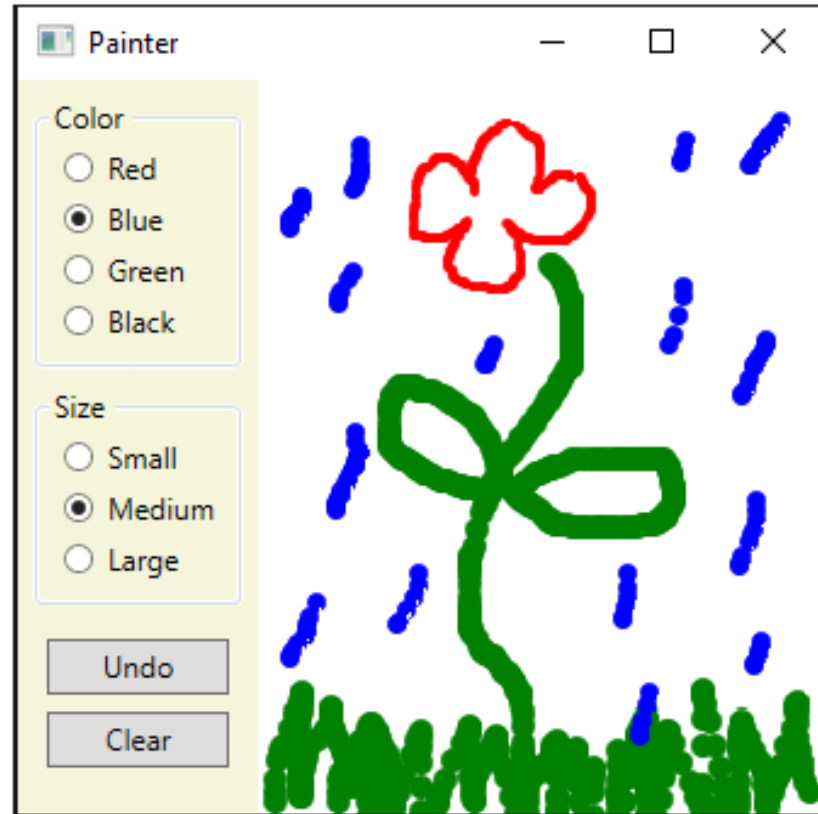


Fig. 1.13 | Drawing rain drops with a medium blue brush.